



SEN381 MILESTONE 1

Armand Johannes Vermeulen 602636
Ditheto Lesiba Phaho 601133
Riegardt Christopher Weich 602547

CivicConnect — PED v1.0

Sections: Problem & Business Need | Stakeholder Analysis | Scope Baseline | Constraints

1. Problem & Business Need

1.1 Current Situation

The organisation currently manages community service requests — facility faults, damaged equipment, security concerns, IT support, maintenance issues, lost property, and other operational requests — through an uncoordinated mix of email, phone calls, WhatsApp messages, spreadsheets, and paper records. Because these channels are not connected to a single controlled record, the organisation faces several recurring problems:

- Requests are duplicated, missed, or sent to the wrong person because there is no single intake point.
- Requesters have no reliable way to see whether a request was received, assigned, delayed, resolved, or closed.
- Staff struggle to prioritise work, know who owns a request, or coordinate with colleagues on the same issue.
- Accountability is weak — status changes and actions taken are not consistently recorded against a request.
- Management lacks a reliable, real-time picture of outstanding, overdue, or resolved work, and has no consistent basis for reporting.
- Potentially sensitive request information is handled inconsistently across informal, unaudited channels.

These symptoms are consistent with the failure modes identified in the IT service management literature when request handling is not treated as a formally managed practice with defined ownership, status tracking, and standardised fulfilment paths (AXELOS, 2020).

1.2 Business Need

The organisation needs a controlled digital platform that gives it a single, traceable, and usable way to submit, manage, monitor, and report on service requests. The solution must improve visibility and accountability without creating a technical, operational, or financial burden the organisation cannot sustain (AXELOS, 2020).

1.3 Business Value by Stakeholder Group

- **Requesters** gain visibility and confidence that a request has been received and is being acted on.
- **Staff** get a single prioritised worklist instead of the current scatter of emails, WhatsApps, and spreadsheets.
- **Management** gains reliable, auditable data to identify overdue work and evaluate service performance.

- **The organisation as a whole** gains a documented, consistent lifecycle for every request, replacing informal and inconsistent handling of information that may be sensitive.

These value statements are the primary justification for the functional and non-functional requirements contained in this baseline.

2. Stakeholder Analysis

Stakeholders below are additionally rated by relative power and interest using Mendelow's power–interest framework, a well-established technique for classifying stakeholders by their ability to influence a project and their level of interest in its outcome (Mendelow, 1991).

Stakeholder	Role / Description	Key Needs & Expectations	Influence	Interest	Example Requirement Derived
Requester	Community member or staff member submitting a service request (e.g. reporting a facility fault or lost property)	Easy submission, visibility of status, confidence the request will not be lost	Low–Medium (numerous, but individually low leverage)	High	The system shall allow a requester to view the current status of any request they submitted.
Front-line Staff	Staff who action requests (maintenance, IT, security, etc.)	Clear, prioritised worklist; ability to claim or assign ownership; a way to record actions without creating duplicate admin work	Medium–High	High	The system shall allow staff to filter requests by category, status, and priority.
Management / Oversight	Supervisors and organisational leadership	Reliable reporting on open, overdue, and resolved requests; accountability data; confidence the platform does not create new risk	High	High	The system shall provide a report of open and overdue requests by category.
System Administrator	Manages accounts,	Controlled way to manage user	Medium	Medium	The system shall allow an

(may be a staff role)	categories, and access control	roles and categories without needing developer involvement			administrator to manage the controlled category list.
Project Sponsor / Lecturer (assessor)	Governs project constraints and evaluates the engineered baseline	Evidence of a controlled engineering process, not just working software	High	High	The team shall maintain a controlled, versioned PED and RTM that demonstrate an auditable engineering process.
Development Team (self)	Builds and maintains the system within schedule and skill constraints	Requirements that are achievable within a 3-person team and the module timeline	Medium	High	Requirements shall be prioritised and sized so that the committed M1–M4 scope remains achievable by a three-person team within the fixed milestone dates.

2.1 Meaningful Conflicts / Competing Expectations

1. Requester visibility versus staff workload.

Requesters want full transparency and fast turnaround on every request. Staff need to triage and prioritise, which means some requests will sit untouched for a period. We therefore treat status transitions (Submitted → Acknowledged → In Progress → Resolved) as the mechanism that gives requesters honest visibility without implying every request is worked on immediately.

2. Management reporting versus staff and requester privacy.

Management wants enough detail to assess performance and accountability. Staff notes on a request (and details submitted by the requester) may include sensitive information that should not be broadly visible. Role-based access is the practical resolution: management sees status and performance data, not necessarily every internal comment.

3. Speed of delivery versus controlled process.

The teaching-team stakeholder (assessor) requires a controlled, auditable engineering process (protected branches, two-reviewer pull requests, and full traceability). Left unmanaged, schedule pressure from other stakeholders could push the team to bypass these controls. This conflict is recorded explicitly so it can be treated as a known risk rather than discovered late. It also reinforces the need for the GitHub governance rules already required by the Master Project Brief.

3. Scope Baseline

The scope statements below are structured as a scope baseline in the project-management sense: an approved definition of what is and is not included, used as the fixed point of comparison for any later change request rather than something that can be informally renegotiated (Project Management Institute, 2017).

3.1 In Scope (M1 commitment, to be built in later milestones)

- **Requester:** submit a new request with category, description, and relevant details; view status and history of own requests; receive feedback when a request's status changes.
- **Staff:** view requests relevant to their role; search, filter, and sort by category, status, and priority; view full request detail; assign or accept ownership; update status through controlled transitions; add comments or resolution notes; resolve or close requests where authorised.
- **Management:** view service activity (open, overdue, resolved, closed); filter by category and status; access enough data to support accountability and performance reporting.
- Role-based access control that distinguishes Requester, Staff, and Management/Admin roles.
- A controlled, fixed category taxonomy for classifying requests (not free text).
- An auditable status and history trail for every request.

These capabilities are expressed at the business-capability level. The precise, testable functional requirements and acceptance criteria are owned by Person 2 and traced in the RTM.

3.2 Out of Scope (for this project)

- Native mobile applications (a mobile-responsive web interface is assumed to meet requester and staff needs).
- Real-time chat or live messaging between requester and staff.
- Payment or billing functionality.
- Integration with external third-party or municipal systems (for example GIS or external ticketing platforms).
- Multi-language localisation.
- AI-assisted automatic categorisation or triage of incoming requests.

3.3 Deferred / Future Scope

- SLA-based automatic escalation of overdue requests.
- Advanced analytics or predictive reporting for management.
- Automated notification via SMS or email gateways beyond in-app status updates (may be revisited once a technology and cost decision is made in M2).
- A public-facing transparency or open-data view of aggregate service performance.

3.4 Defending a Deliberate Exclusion — AI-Assisted Categorisation

AI-based automatic categorisation was deliberately excluded from scope rather than left ambiguous.

- **Why it matters now:** it directly affects the data model and the requester submission requirement (does the requester choose a category, or does the system infer one?).

- **What is missing:** the team has no historical request data to train or validate a classifier, and no evidence yet that manual categorisation is actually a bottleneck.
- **Risk of doing it anyway:** misclassification would undermine staff trust in the platform faster than the manual step it replaces, and would consume schedule and cost budget that is disproportionate to its value in a class-project timeframe.
- **Decision:** requesters select from a controlled category list at submission (manual, but structured). Automatic categorisation is recorded as future scope and will only be revisited if clear evidence of a real bottleneck emerges.

4. Constraints Analysis

Constraint	Description (CivicConnect-specific)	Engineering Implication
Scope	The committed scope in Section 3.1 must be baselined. Any addition goes through the team's change-control process, not informal agreement.	Requirements and the RTM must be stable enough to support a controlled M1 sign-off. Scope-creep risk must be tracked in the Risk Register
Schedule	The project must progress through four fixed milestones within the SEN381 delivery period. M1 has a hard presentation date.	Favours a smaller, fully controlled feature set over a larger, partially finished one. This directly supports the deferrals recorded in Section 3.3.
Cost / Resources	Team of exactly three students. Prefer free or low-cost services, but identify their real-world operational limits.	Any technology or hosting decision made in M2 must record free-tier limits as an explicit assumption that will be revisited, because those limits constrain later architecture and operational options.
Quality	Quality attributes (for example reliability of status updates and usability for non-technical requesters) must be defined now and evidenced later.	Non-functional requirements from Person 2 must be measurable (for example "status updates are visible to the requester within X seconds of the staff action") so that M3 testing has something concrete to verify against (ISO/IEC, 2011).
Security	Service requests may contain sensitive information (for example security concerns or personal details). Handling must be considered from M1, not added later. This includes an obligation to process personal information	Role-based access control is treated as an M1-identified requirement, not an M2 afterthought. This feeds both the RBAC requirements (Person 2) and the Forward Engineering Considerations and Risk Register (Person 3). RBAC follows the widely adopted NIST/ANSI reference model for assigning permissions through roles rather than to individual

lawfully under South Africa's
Protection of Personal Information
Act (Republic of South Africa, 2013).

users directly (Sandhu, Ferraiolo and Kuhn, 2000;
American National Standards Institute/INCITS,
2004).

4.1 Constraint Interaction / Trade-off

Schedule versus Quality and Security.

The fixed three-person team and fixed timeline interact directly with the quality and security constraints. If the team had committed to broader scope now (for example SLA escalation or notification integrations), the time needed to properly test and secure those features would compress the time available to implement and verify the core RBAC and audit-trail requirements that management and security stakeholders actually need. The scope decisions in Section 3 are a direct response to this trade-off: protecting the quality and security evidence for the core capabilities was prioritised over breadth of features.

Requirements Baseline.

5.1 Requirements approach:

The requirement baseline converts the stakeholder expectations and business needs listed in sections 1-4 into precise and verifiable system requirements. Non-functional requirements specify how the system should function and what quality attributes it should contain, while functional requirements relate more to what the CivicConnect system should allow its users to do.

To facilitate traceability across the engineering lifecycle, requirements will be given distinct identities. FR-XXX will be used to refer to functional requirements, whilst NFR-XXX will be used to refer to non-functional requirements and AC-XXX to the acceptance criteria. The acceptance criteria will serve as the condition for the verification of requirement completion.

5.2 Functional requirements:

ID	REQUIREMENT	STAKEHOLDER	PRIORITY	ACCEPTANCE CRITERIA	AC-ID
FR-001	The system must allow authorised users to submit new service requests containing all relevant information	Requester	High	System prevents submission of request when required requester & request information is missing. Valid submission creates new service request.	AC-001
FR-002	The system allows users to categorise their requests from a list of available service categories.	Requester, staff and management	High	User can select category from available list. Submission is denied without categorisation.	AC-002
FR-003	The system must assign unique identifiers for all successful service requests	Requester	High	Each successful request submission receives unique ID which can be used to identify and manipulate request for further use.	AC-003

FR-004	The system shall allow authorized users to view the status of their submitted service requests.	Requester	High	Users can view requests they've made. Users can see the status of requests relevant to them.	AC-004
FR-005	The system will allow users to view their submitted service request history.	Requester	High	Previously submitted requests can be viewed along with relevant details in request history section.	AC-005
FR-006	The system will provide meaningful system-level feedback to users when service requests are accepted, rejected, updated or fulfilled.	Requester	High	Users receive relevant update or notification when request status is changed.	AC-006
FR-007	The system shall allow authorised staff to view service requests relevant to their roles and permissions.	Staff	High	Users can view relevant requests. Requests outside authorised access are unavailable.	AC-007
FR-008	The system shall allow authorised staff to filter, search and/or sort service requests using attributes of the relevant request.	Staff	High	Authorised staff can filter through at least 1 category selection. Results shown match selected attribute/s.	AC-008
FR-009	The system will allow staff to view the full details pertaining to their service requests.	Staff	High	System displays selected request as well as all its attributes and details.	AC-009
FR-010	The system will allow authorised staff to assign and/or accept responsibility of service requests.	Staff	High	Unauthorised staff can not modify responsibility assignments. Assigned staff is recorded with relevant request.	AC-0010
FR-011	The system will allow authorised staff to update service requests through controlled and authorised processes.	Staff	High	Authorised staff can update request status. Only authorised status transitions recorded.	AC-011
FR-012	The system will allow authorised staff to track relevant actions, comments and resolution information of relevant service requests.	Staff	High	Authorised staff can select request and track relevant actions, comments and resolution status.	AC-012
FR-013	The system will allow authorised staff to resolve or close service requests relevant to their roles or responsibilities.	Staff	High	Authorised staff can mark service request as resolved or closed. Requester can see relevant change on request.	AC-013
FR-014	The system will allow authorised management to view useful service request activity information.	Management	High	Authorised management can view major changes in service requests e.g. Ownership change.	AC-014

FR-015	The system shall allow authorised management to identify open, overdue, closed and resolved requests.	Management	High	Authorised management can view current service activity and statuses.	AC-015
FR-016	The system will allow authorised management users to view service requests by category, status and/or other criteria relevant to service requests.	Management	Medium	Authorised management can view service requests by at least one attribute.	AC-016

5.3 Non-functional requirements:

ID	QUALITY ATTRIBUTE	NON-FUNCTIONAL REQUIREMENT	STAKEHOLDER	PRIORITY	ACCEPTANCE CRITERIA	AC-ID
NFR-017	Security	The system will ensure that before access to functionality and requested information is given, users are authorised.	All stakeholders	High	Authorised users are granted permission in accordance with their roles. Unauthorised users cannot access functionality.	AC-017
NFR-018	Authorisation	Role-based authorisation is used by the system so that users can only perform actions suitable for their respective roles	All stakeholders	High	An authenticated user performing an action outside of their allocated role rights is denied access.	AC-018
NFR-019	Data security	Sensitive request information will only be made available to staff with the relevant authorisation.	Requester/Staff	High	Retrieval of request information is declined without proper authorisation.	AC-019
NFR-020	Auditability	Major changes to the service requests will be tracked by the system.	Management	High	A change in the staff member responsible for a particular service request or status will be recorded.	AC-020
NFR-021	Usability	The system will provide clear labels, validation and feedback to users for service request submission as well as subsequent information and changes pertaining to the request.	Requester	High	Mandatory fields are clearly labelled. Invalid or incorrect submissions receive appropriate feedback. Successful submissions produce clear confirmation.	AC-021
NFR-022	Reliability	The system will successfully store service request information during regular system operations.	All stakeholders	High	Successful service request submissions remain retrievable after submission is complete. Information remains stored with corresponding ID.	AC-022

NFR-023	Maintainability	The system will be built using modular components to allow changes to one component to remain isolated.	All stakeholders	High	Major system components will be well defined responsibilities. Code is reviewed to ensure future modification isn't impossible.	AC-023
NFR-024	Performance	The system will give users feedback for regular operations within 3 seconds for at least 95% of requests.	All stakeholders	High	Performance testing demonstrates that at least 95% of requests receive feedback within the 3 second target.	AC-024

6. Initial requirements traceability matrix:

STAKEHOLDER	REQUIREMENT ID	ACCEPTANCE CRITERIA	PRIORITY	TRACEABILITY STATUS	Design (TBD-M2)	IMPLEMENTATION (TBD-M3)
Requester	FR-001-006, NFR-019, NFR-021	AC-001-006, AC-019-021	High	Baseline		
Staff	FR-007-013, FR-002, NFR-019	AC-007-013, AC-002, AC-019	High	Baseline		
Management	FR-014-016, FR-002, NFR-020	AC-014-016, AC-002, AC-020	High	Baseline		
All stakeholders	NFR-017-018, NFR-022-024	AC-017-018, AC-022-024	High	Baseline		

Engineering Governance & Project Control

7. Risk Register

The risk assessment method for the following risk followed a 5-step risk assessment lifecycle, which includes identifying analysing, evaluating, mitigating, and monitoring (PMI, 2021).

ID	Risk	Probability	Impact	Priority	Mitigation	Owner	Contingency	Status
R1	Stakeholder requirements change after baseline	High	High	Critical	Formal change control	Team	Re-baseline scope after approval	Open
R2	Security requirements overlooked	Medium	High	High	Early NFR validation	Team	Conduct a security review and update affected requirements with approval	Open
R3	Poor Git collaboration causes conflicts	Medium	Medium	Medium	Feature branches + PRs	Repository owner	Assign Conflict resolution Lead to manage any conflicts	Open
R4	Schedule delays from unequal workload	Medium	Medium	Medium	Weekly progress reviews	Team	Reassign critical tasks to team members and adjust milestone schedule	Open

Risk assessment method and reasoning

Identify Risks: This step requires brainstorming by assessing the environment, conditions and outcomes of the project. The main attributes of the project is: Digital, Team collaboration, Security, and Performant. For example: Since the system is inherently digital it is expected to be easy to change, this could cause stakeholder expectations to allow for changes in the baseline (PMI, 2021).

Evaluate & Prioritize: After identifying the risks, the team can agree on what risks have the highest probability and impact. Combining the likelihood and the consequences of each, a priority is assigned to each risk. This is to rank the risks on the net negative they could cause the project, symbolised by the priority of the risks (PMI, 2021).

Mitigating Risks: This step involves if the risk is to be avoided, transferred, reduced or accepted. A plan for each risk is created to optimistically try and reduce each risk. As well as an contingency for if the risk occurs. (PMI, 2021)

Monitor & Review: After the risk register is created constantly track each risk at each weekly progress review and update mitigation strategies as the conditions of the project change. Risk status is a column that represents if the risk is still a possibility and will be closed if all causalities are eliminated (PMI, 2021).

8. Forward Engineering Considerations

The following six considerations will impact current decisions and should be accounted for to conserve good engineering practices.

Consideration	Present	Future Influences	Missing Information	Risk if ignored
Security	Project Requirements must prioritize user data, to ensure legal responsibility (NIST, 2020)	Implementing Authentication when a user logs in, authorization when the user interacts with the application	Exact technology and hosting security controls.	Data exposure and lost of trust.
Scalability	Expected user growth affects current requirements	Appropriately managing scaling when traffic is high and building the system architecture to be able to handle more users	Expected user volumes/traffic.	Performance degradation and costly redesign
Testability	Project deliverables should be easily testable and measurable to set a baseline for future implementations. (ISO/IEC 25010, 2011)	Affects test design, confirming tests coverage is sufficient to cover all possible use cases	Detailed test environment definition	Requirements become difficult to verify
Deployment	Deciding the appropriate hosting service for NFRs (Bass, Weber & Zhu, 2015)	Ensuring the infrastructure of the deployment service is sufficient for the performance demands.	Final hosting platform	NFRs may become unattainable
Data recovery	Backups of all reports must be made to ensure data integrity and keep (NIST, 2020)	Affecting backup strategy and workflows to ensure all data is saved and preserved for future use.	Retention and backup frequency requirements.	Permanent important data loss.
Observability	Setting up tracking methods and plans for the services the project could use (Bass, Weber & Zhu, 2015)	Monitoring and logging any issues the live project runs into after deployment and in deployment testing.	Monitoring tooling choices.	Live environment issues may go undetected.

These considerations are not implementation methods rather guidelines to display how system design and choices are impacted by future and current considerations and conditions.

9. Engineering Decision Log

Decision logs for Milestone 1, justifying why decisions are deferred or accepted

Decision ID	Decision	Status	Reason
D1	Use GitHub Controlled repository.	Accepted	Required for source control and governance.
D2	Deferring technology stack choice.	Deferred	To preserves architectural flexibility (Sommerville, 2016).
D3	Deferring Database Design decisions	Deferred	To ensure future influences won't cause delays in development. (Sommerville, 2016).
D4	Requirements IDs follow Functional /Non-Functional Requirements numbering	Accepted	IDs assigned to support traceability. (Bass, Weber & Zhu, 2015)

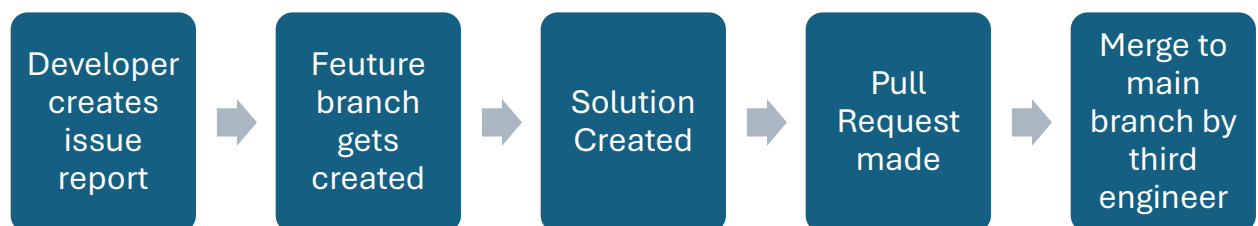
10. GitHub Governance

GitHub Repository Name: SEN381-Project

Team members: Armand Vermeulen (Armand-JV), Riegardt Weich (xStoffer), Ditheto Phaho (Deee-dot)

Branch Strategy: Each branch should start with a descriptor of what its achieving and then the feature/bugfix ID or name. For example: bugfix/issue-123 or feature/Login-Frontend (Conventional Branch, 2026)

Pull request workflow:

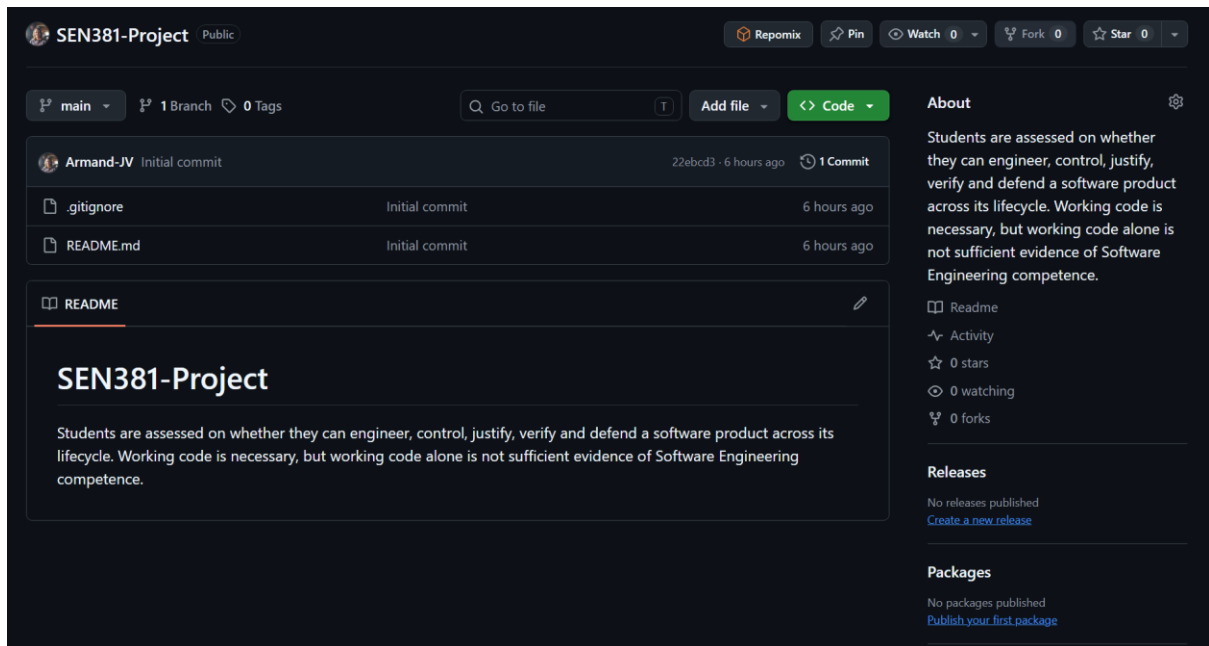


The main branch is protected by blocking direct pushes, requiring pull requests, required status checks by using automated tests and engineers can't delete branch history (GitHub, 2026).

GitHub link : <https://github.com/Armand-JV/SEN381-Project>

10.1 Evidence checklist

☒ Repository created



☒ Protected main branch

Restrict deletions: Only allow users with bypass permissions to delete matching refs. (GitHub, 2026)

Require linear history: Prevent merge commits from being pushed to matching refs. (GitHub, 2026)

Require a pull request before merging: Require all commits be made to a non-target branch and submitted via a pull request before they can be merged. (GitHub, 2026)

- Required approvals: 2(GitHub, 2026)
- Dismiss stale pull request approvals when new commits are pushed. New, reviewable commits pushed will dismiss previous pull request review approvals. (GitHub, 2026)
- Require approval of the most recent reviewable push. Whether the most recent reviewable push must be approved by someone other than the person who pushed it. (GitHub, 2026)

Block force pushes: Prevent users with push access from force pushing to refs. (GitHub, 2026)

Branch rules

☐

Restrict creations

Only allow users with bypass permission to create matching refs.

☐

Restrict updates

Only allow users with bypass permission to update matching refs.

☒

Restrict deletions

Only allow users with bypass permissions to delete matching refs.

☒

Require linear history

Prevent merge commits from being pushed to matching refs.

☐

Require deployments to succeed

Choose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.

☐

Require signed commits

Commits pushed to matching refs must have verified signatures.

☒

Require a pull request before merging

Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.

Show additional settings

☐

Require status checks to pass

Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.

☒

Block force pushes

Prevent users with push access from force pushing to refs.

☐

Require code scanning results

Choose which tools must provide code scanning results before the reference is updated. When configured, code scanning must be enabled and have results for both the commit and the reference being updated.

☐

Require code quality results

Choose which severity levels of code quality results should block pull request merges. When configured, a code quality analysis must be done on the pull request before the changes can be merged.

☐

Restrict code coverage

Enforce minimum line coverage thresholds on pull requests. When configured, uploaded coverage data must meet the specified criteria before changes can be merged.

☒

Automatically request Copilot code review

Request Copilot code review for new pull requests automatically if the author has access to Copilot code review and their premium requests quota has not reached the limit.

Show additional settings

☒ Feature branches used

Overview					
Yours					
Active					
Stale					
All					
Q Search branches...					
Default					
Branch	Updated	Check status	Behind	Ahead	Pull request
main	6 hours ago			Default	
Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
Armand-JV-patch-1	4 minutes ago		0	1	#1
feature/Engineering-Governance	9 minutes ago		0	0	
Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
Armand-JV-patch-1	4 minutes ago		0	1	#1
feature/Engineering-Governance	9 minutes ago		0	0	

☒ Pull Requests recorded

Add initial content for Engineering Governance #1

Awaiting approval

Code

Open

Armand-JV wants to merge 1 commit into `feature/Engineering-Governance` from `Armand-JV-patch-1`

Conversation 0

Commits 1

Checks 0

Files changed 1

Armand-JV commented 3 minutes ago • edited

Owner

GitHub Governance checks

20cb622

Verified

Add initial content for Engineering Governance

Armand-JV self-assigned this now

Armand-JV added the `documentation` label now

Armand-JV added this to the `Milestone 1` milestone now

Review required

At least 2 approving reviews are required by reviewers with write access.

Merging is blocked

New changes require approval from someone other than the last pusher.

☐ Merge without waiting for requirements to be met (bypass rules)

Squash and merge

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

Add a comment

Reviewers

No reviews—at least 2 approving reviews are required.

Still in progress? [Convert to draft](#)

Assignees

Armand-JV

Labels

`documentation`

Projects

None yet

Milestone

Milestone 1

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Unsubscribe

You're receiving notifications because you were assigned.

1 participant

☒ Peer reviews documented

Add initial content for Engineering Governance #1

Merged

xStoffer merged 1 commit into `feature/Engineering-Governance` from `Armand-JV-patch-1` 2 minutes ago

Conversation 2

Commits 1

Checks 0

Files changed 1

Armand-JV commented 44 minutes ago • edited

Owner

GitHub Governance checks

20cb622

Verified

Add initial content for Engineering Governance

Armand-JV self-assigned this 42 minutes ago

Armand-JV added the `documentation` label 42 minutes ago

Armand-JV added this to the `Milestone 1` milestone 41 minutes ago

Deee-dot approved these changes 22 minutes ago

View reviewed changes

Deee-dot left a comment

Collaborator

Reviewed the changes. The commit is clear and the proposed change is consistent with the purpose of the feature/GitHub-Governance branch. No issues found. Approved.

xStoffer commented 6 minutes ago

Collaborator

I have independently reviewed the proposed changes and the commit. The change is understandable, appropriate for Milestone 1, and no issues were identified. Approved for merging.

Reviewers

xStoffer ✓

Deee-dot ✓

Assignees

Armand-JV

Labels

`documentation`

Projects

None yet

Milestone

Milestone 1

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Unsubscribe

You're receiving notifications because you were assigned.

3 participants

Lock conversation

11. AI Usage Register

Date	Artefact	AI use	Human Verification
07 September	Risk Register	Table Structure	Confirmed project requirements match table structure (Edited)
07 September	Forward Engineering Considerations	Strictly Consideration column	Researched common considerations for similar projects
08 September	Engineering Decision Log	Decision confirmation by AI review	Decisions created by Engineer
09 September	PED v1.0	Identify inconsistencies in the document between drafts	Compared to AI output to Milestone 1 requirements and fixed inconsistencies.
09 September	RTM blueprint	Assistance brainstorming ideas for final RTM table	Compared against master brief and required directives.
09 September	Stakeholder Analysis, Scope Baseline	Assistance in finding sources.	Individually checked the sources.

12. Version History

Version	Date	Author	Changes
0.1	06 September	Armand	Work split and format
0.8	08 September	Team	Each member integrates sections
1.0	09 September	Team	Verified baseline for submission

13. Bassline Sign-off

Team member	Role	Signature	Date
Riegardt Weich	Analysis		09 September
Ditheto Phaho	Requirements		09 September
Armand Vermeulen	Governance		09 September

Sources

American National Standards Institute/INCITS (2004) *INCITS 359-2004: Information technology — role based access control*. Washington, DC: ANSI.

AXELOS (2020) *Service request management: ITIL® 4 practice guide*. London: AXELOS Ltd.

International Organization for Standardization/International Electrotechnical Commission (2011) *ISO/IEC 25010:2011 — Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models*. Geneva: ISO.

Mendelow, A.L. (1991) 'Environmental scanning: the impact of the stakeholder concept', in *Proceedings of the Second International Conference on Information Systems*. Cambridge, MA.

Project Management Institute (2017) *A guide to the project management body of knowledge (PMBOK® guide)*. 6th edn. Newtown Square, PA: Project Management Institute.

Republic of South Africa (2013) *Protection of Personal Information Act, No. 4 of 2013*. Pretoria: Government Printer.

Sandhu, R., Ferraiolo, D. and Kuhn, R. (2000) 'The NIST model for role-based access control: towards a unified standard', in *Proceedings of the Fifth ACM Workshop on Role-Based Access Control*. Berlin, 26–28 July. New York: ACM, pp. 47–63.

Bass, L., Weber, I. and Zhu, L. (2015). *DevOps a software architect's perspective*. Addison-Wesley.

GitHub Docs. (2026). *About protected branches*. [online] Available at: <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches> [Accessed 9 Sept. 2026].

ISO - International Organization for Standardization (2018). *ISO/IEC/IEEE 29148:2018*. [online] ISO. Available at: <https://www.iso.org/standard/72089.html> [Accessed 9 Sept. 2026].

NIST (2020). *Security and Privacy Controls for Information Systems and Organizations*. [online] csrc.nist.gov. Available at: <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final> [Accessed 9 Sept. 2026].

Project Management Institute (PMI) (2021). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. [online] [Pmi.org](https://www.pmi.org). Available at: <https://www.pmi.org/standards/pmbok> [Accessed 9 Sept. 2026].

Sommerville, I. (2016). *Software engineering tenth edition*. [online] Available at: <https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf> [Accessed 9 Sept. 2026].

Conventional Branch. (2026). *Conventional branch — A git branch naming convention*. [online] Available at: <https://conventionalbranch.org/> [Accessed 9 Sept. 2026].